

FireProtect : An IoT - based Smart Fire Detection System

Saisha Verma

Dept. of Computer Science & Engineering
Indira Gandhi Delhi Technical University for Women
Delhi, India
saishaverma0512@gmail.com

Riya Singh

Dept. of Computer Science & Engineering
Indira Gandhi Delhi Technical University for Women
Delhi, India
riya.sin4197@gmail.com

Ritisha Sood

Dept. of Computer Science & Engineering
Indira Gandhi Delhi Technical University for Women
Delhi, India
ritisha2935@gmail.com

Ms. Kavita Sharma

Dept. of Computer Science & Engineering
Indira Gandhi Delhi Technical University for Women
Delhi, India
kavita004phd20@igdtuw.ac.in

Abstract—Fire accidents often result in severe damage due to delayed detection and limited monitoring in conventional systems. This paper presents an IoT-based smart fire detection system integrating MQTT-based communication and machine learning for real-time and low-latency monitoring. Sensor data from multiple environmental parameters is transmitted to a web dashboard using a publish-subscribe model. A hybrid approach combining Decision Tree (edge-level) and Random Forest (application-level) enables accurate classification into actual fire, false alarm, and no fire. The system achieves fast response and reliable performance, making it suitable for real-time fire monitoring applications.

Index Terms—Internet of Things (IoT), Smart Fire Detection, MQTT Protocol, Machine Learning, Real-Time Monitoring, Edge Computing, Random Forest, Decision Tree, Fire Classification, Emergency Response System, Web Dashboard, Smart Safety Systems

I. INTRODUCTION

Across India, flames still claim homes and lives alike, whether in cities or farmland. Every year, NCRB reports many people killed by fire, with Odisha leading at 1,032 in 2023 while Maharashtra saw 697 and Tamil Nadu counted 616 [1]. Much of these losses come from faulty wiring or avoidable conditions. At the same time, more than 6.3 lakh forest fires were spotted between 2021 and 2024, showing how broad the risk really is [2]. Local detection is what traditional fire alarms offer so far, yet they fail to send instant updates from distant locations or link broader data networks like weather patterns or rescue routing [3].

As cities grow faster, so do the dangers waiting inside them. A fresh idea takes shape here: sensors watch air quality, temperature, gas concentration, humidity, and flame levels nonstop. Data flows through MQTT-based communication and cloud platforms, turning readings into live graphs and triggering alerts only when real-time conditions demand attention [4]. Once danger is confirmed, automated messages reach the closest fire station exactly where flames burn, reducing response delay and improving rescue efficiency. What stands

out are fire detection systems powered by IoT, linked to the cloud, supported by machine learning-based validation, and capable of sending alerts directly to nearby stations without human intervention [5].

II. LITERATURE REVIEW

In recent years, advancements in IoT technologies, cloud platforms, and intelligent data processing have significantly improved fire detection systems. Traditional alarms, which rely on isolated smoke or heat sensors, often struggle with complex scenarios and generate false alerts, particularly in unattended environments. These shortcomings have encouraged the development of connected systems capable of continuous monitoring and adaptive response.

Earlier work, such as that by Debnath's group, presents IoT-based frameworks where multiple sensors monitor environmental conditions and transmit data using the MQTT protocol for remote access and alert generation. While these systems support automation, they still face challenges related to real-time data handling, communication reliability, and accurate location-based response. Other approaches employ NodeMCU-based architectures and rule-based classification to reduce false alarms [9], [11]. Additionally, deep learning models such as FireNet and MobileNet [5], [6], along with image-based detection methods [7], [8], [10], have shown improved accuracy but require higher computational resources and may not guarantee low-latency alert delivery.

Recent research has focused on integrating machine learning with IoT systems to enhance decision-making. Multi-sensor data is used to train classification models that distinguish between actual fire, false alarms, and normal conditions. However, many existing implementations depend heavily on cloud processing, introducing latency, or lack efficient real-time communication mechanisms for immediate alert propagation.

To overcome these limitations, the proposed system combines a hybrid machine learning framework with MQTT-based

TABLE I: Comparison of Existing IoT-Based Fire Monitoring Systems

Ref.	System Type	Sensors / Data Source	ML / DL	Cloud	Location	Communication	Emergency Authority	Key Limitations
[1]	IoT-based fire severity monitoring system	Smoke and temperature sensors	No	Yes	No	MQTT	Partial (user alerts only)	No automatic coordination with fire stations
[2]	Smart home fire alert system	Temperature and gas sensors	No	Yes	No	HTTP / Cloud	No	Limited to local user notifications
[3]	Smart city fire notification system	Environmental sensors	No	Yes	Yes	Cloud-based	Yes	Lacks cloud-based analytics and visualization
[4]	ML-based fire surveillance system	Heat, smoke, and flame sensors	Yes (ML)	Limited	No	Not specified	Partial	No real-time cloud dashboard
[5]	DL-based surveillance fire detection system	CCTV video streams	Yes (CNN, MobileNet)	Yes	Yes	Streaming / Cloud	Partial	High computational requirements
[6]	Deep learning-based fire detection (FireNet)	Image and video datasets	Yes (FireNet, transfer learning)	No	No	Offline	No	No IoT integration or real-time alert mechanism
[7]	Multimodal indoor fire detection system	Sensor data and image processing	Yes (ML/DL)	Yes	No	Mixed	No	No emergency service reporting mechanism
[8]	Big data-based house fire detection system	Large-scale sensor data	Yes (data analytics)	Yes	No	Cloud	No	No real-time response or alert mechanism
Proposed System	IoT-based real-time fire detection system with MQTT and ML	Gas, temperature, humidity, PIR, flame sensors	Yes (Hybrid ML: DT + RF)	Yes	Yes	MQTT (real-time)	Yes	Dependent on network connectivity; public broker limitations

communication using a public broker such as HiveMQ. A lightweight Decision Tree model is deployed at the edge for real-time prediction and actuation, while a Random Forest model ensures high-accuracy classification at the application level. This integration enables low-latency data transmission, reduces false alarms, and improves overall system reliability, while fallback cloud support maintains robustness and scalability.

III. PROPOSED SYSTEM

IoT-based fire detection and real-time alert system designed for early detection and rapid emergency response. The system integrates environmental sensing, MQTT-based communication and live web-based visualization to overcome the latency and limitations of conventional cloud-polling fire alarm systems. It enables real-time monitoring through a publish-subscribe model, where sensor data is instantly transmitted to a public broker and delivered to the web dashboard without delay. By combining low-cost sensors, an ESP32 microcontroller, a public MQTT broker, and a responsive frontend, the system ensures fast detection, minimal latency, and improved situational awareness. Additionally, the

incoming sensor data is processed using a lightweight machine learning model to distinguish between true fire incidents and false alarms, enhancing detection accuracy and reducing unnecessary alerts.

A. System Architecture

- 1) **Sensing Layer:** The sensing layer is responsible for real-time acquisition of environmental parameters associated with fire hazards. It consists of an ESP32 interfaced with multiple sensors, including the MQ-2 Gas Sensor for detecting combustible gases, a Flame Sensor Module for flame detection, a PIR Sensor for human presence, and a DHT11 for monitoring temperature and humidity. These sensors continuously generate analog and digital signals, which are sampled by the ESP32 at regular intervals. The combination of multiple sensors improves reliability by enabling multi-parameter fire detection rather than relying on a single indicator.
- 2) **Edge Layer:** The edge layer performs local processing and immediate decision-making. The ESP32 reads raw sensor values and applies threshold-based filtering along with a lightweight machine learning model. A

TABLE II: Hardware Specifications

Component	Model / Type	Description	Supply Voltage
Microcontroller	ESP32 Development Module	Acts as the central processing unit with integrated Wi-Fi for sensor data acquisition and cloud communication	3.3 V / 5 V
Temperature Sensor	DHT11	Measures ambient temperature and humidity to detect abnormal heat rise	3.3 V–5 V
Gas Sensor	MQ-2	Detects combustible gases and smoke for early fire indication	5 V
Flame Sensor	IR-Based Module	Senses infrared radiation emitted by open flames	3.3 V–5 V
PIR Sensor	HC-SR501	Detects human motion in the monitored area	5 V
Buzzer	Active Buzzer	Provides an immediate audible alert during hazardous conditions	3.3 V–5 V
Relay Module	Single-Channel Relay	Controls external safety devices such as water pumps or exhaust systems	5 V
Power Supply	Regulated USB Supply	Provides stable power to all hardware components	5 V

Decision Tree classifier is converted into rule-based logic (if–else conditions) for efficient execution on the microcontroller. Based on sensor inputs such as high gas concentration, elevated temperature, and flame presence, the system classifies the situation into “true fire,” “false alarm,” or “no fire.” In the event of a true fire, a buzzer or relay module is activated instantly, ensuring rapid on-site response even without internet connectivity. This layer reduces latency and enables real-time actuation.

- 3) **Networking Layer using MQTT Protocol:** For real-time communication, the system uses the MQTT protocol, a publish–subscribe messaging model optimized for IoT applications. The ESP32 acts as a publisher, sending structured JSON data (sensor readings and alert status) to topics such as `firealarm/data` and `firealarm/alert`. These messages are transmitted via the HiveMQ over secure WebSockets. The web application acts as a subscriber, connecting to the broker and listening to these topics. Upon receiving messages, the website parses the data and updates the user interface in real time. This architecture ensures low-latency, scalable, and efficient communication without direct coupling between devices.
- 4) **ThingSpeak Cloud as Fallback:** To enhance system reliability, the ThingSpeak cloud is integrated as a secondary data channel. The ESP32 periodically uploads sensor data to ThingSpeak using HTTP APIs. In cases where the MQTT connection fails due to network issues, the web application retrieves the latest data from the cloud at fixed intervals. Additionally, ThingSpeak provides data logging, visualization, and historical analysis capabilities, which are useful for monitoring long-term trends and system performance.
- 5) **Application Layer:** The application layer provides visualization, alerting, and decision support. The web dashboard displays live sensor readings, fire classification results, and alert status in real time. A machine learning model (Random Forest) deployed on the backend further

validates incoming data to improve prediction accuracy and reduce false alarms. When a fire event is detected, instant notifications are generated and displayed on the interface. Integration with Maps API enables geolocation-based visualization of the incident, allowing users or authorities to identify the exact location of the fire. This layer ensures rapid situational awareness and supports timely emergency response.

B. Framework and Integration

The proposed system integrates a modern web application framework, real-time communication protocol, machine learning pipeline, and application services to enable efficient fire detection and monitoring.

- 1) **Hardware and Edge Integration:** Environmental sensors are interfaced with the ESP32 microcontroller for real-time data acquisition. Embedded C/C++ (Arduino framework) is used to process sensor inputs and implement rule-based Decision Tree logic for immediate fire detection and buzzer activation.
- 2) **Website Framework:** The frontend is developed using React.js with TypeScript and Vite for fast rendering and modular development. The component-based architecture includes reusable UI modules such as sensor cards, alert panels, and visualization charts. Custom React hooks (e.g., `useMqtt`, `useToast`) manage state, real-time data flow, and notifications. Backend integration is handled using Supabase services for data management and serverless functions.
- 3) **Communication Protocol:** The system uses the MQTT protocol over secure WebSockets for real-time data transmission. The ESP32 publishes sensor data to topics such as `firealarm/data` and `firealarm/alert`, which are routed through the HiveMQ broker. The web application subscribes to these topics using the `mqtt.js` library, enabling instant updates without polling.

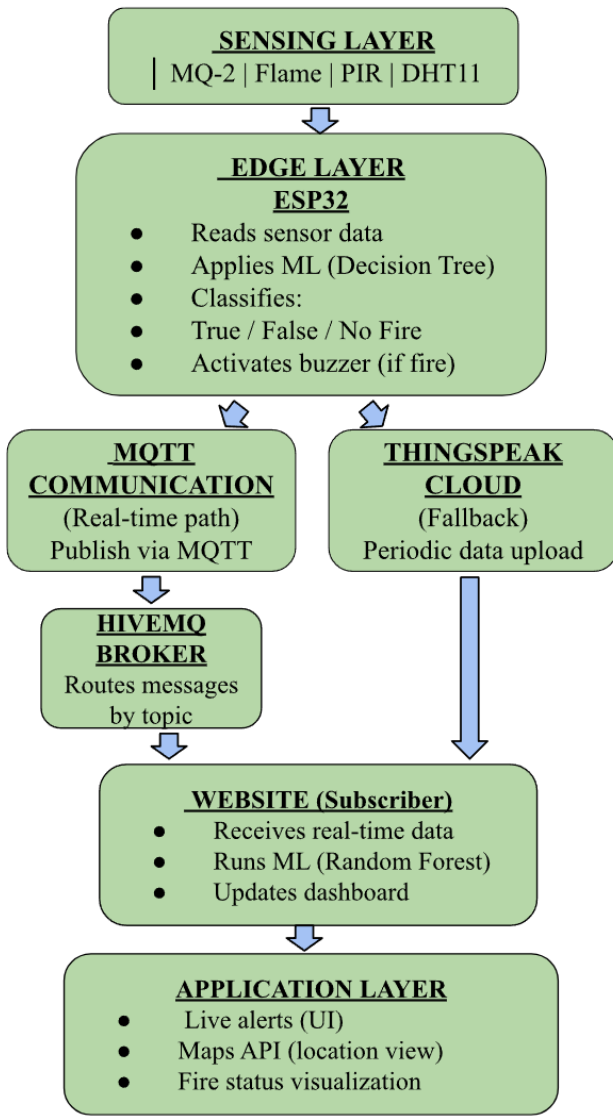


Fig. 1: System Architecture

4) **Machine Learning Framework:** The machine learning pipeline is developed using Python-based tools, with models trained offline and exported for deployment. Data processing and analysis are performed using libraries such as `pandas` and `numpy`, while visualization of class distribution, feature relationships, and model performance is carried out using `matplotlib` and `seaborn`. The machine learning models are implemented using the `scikit-learn` framework, which provides utilities for preprocessing, training, and evaluation. Techniques such as label encoding, train-test splitting, feature scaling using `StandardScaler`, and class balancing using `SMOTE` are applied to ensure robust model performance. A Random Forest model (stored as `rf_model.json`) is integrated into the backend for prediction, enabling accurate classification of incoming

sensor data into fire categories and supporting intelligent alert generation. Additionally, a lightweight Decision Tree model is trained and converted into rule-based logic for deployment at the edge on the ESP32, enabling real-time actuation. Model evaluation is performed using metrics such as accuracy, precision, recall, and F1-score, along with confusion matrix visualization and cross-validation to ensure reliability and generalization.

5) **Application Services:** The system includes multiple service modules such as alert management, fire prediction, and ThingSpeak integration. The alert manager processes incoming events and triggers notifications, while the ThingSpeak service acts as a fallback data source. Visualization components display real-time sensor readings, alert trends, and system status. Integration with Maps API enables geolocation-based visualization for rapid emergency response.

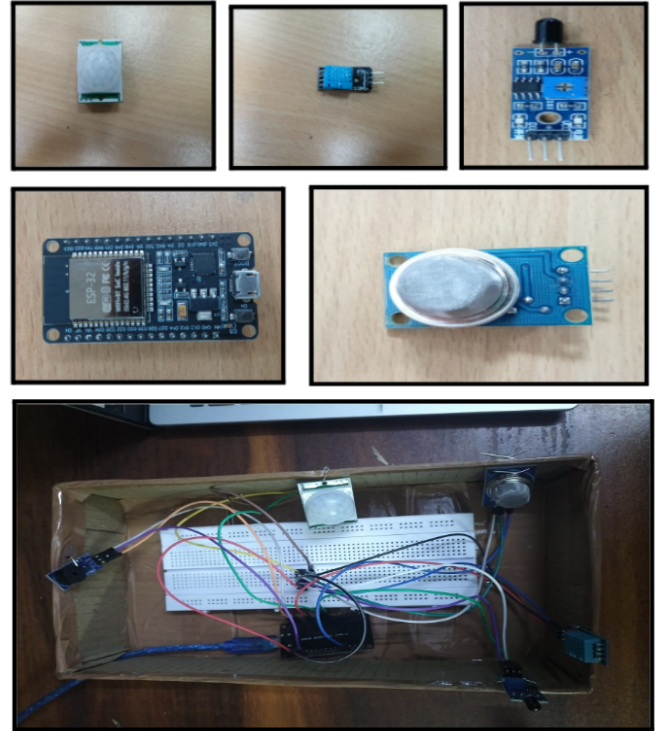


Fig. 2: Hardware Components and Interfaced System

C. Data Acquisition

The dataset was generated using controlled and simulated environmental conditions to represent both fire and non-fire scenarios. It consists of 20,000 samples with sensor readings for gas concentration, temperature, humidity, PIR motion, and flame presence, labeled as actual fire, false fire alarm, or no fire. The data captures a range of conditions, including normal indoor environments, moderate disturbances, and critical fire-like situations.

Data acquisition involved testing different scenarios such as high temperature without fire, gas leakage, and combined

sensor triggers to simulate real fire events. False alarm cases, including appliance heat and human movement, were also included. Key considerations during data generation include:

- Variation of sensor values to avoid feature bias.
- Inclusion of edge cases with partial sensor activation.
- Balanced class distribution for effective learning.
- Simulation of real-world noise and fluctuations.

This approach ensures the dataset supports accurate and reliable fire detection across diverse conditions.

D. Data Storage and Processing

The collected dataset was stored in a structured CSV format, where each record contains sensor readings such as gas concentration, temperature, humidity, PIR motion, and flame detection along with the corresponding fire status label. The data was loaded into the processing environment for initial inspection, including checking data types, missing values, and class distribution. Since the target variable was categorical, label encoding was applied to convert the classes (actual fire, false alarm, no fire) into numerical form. The dataset was then split into input features and target output, followed by an 80:20 train-test split using stratification to maintain class balance across both sets.

Further preprocessing steps were applied to prepare the data for model training and improve performance:

- Handling class imbalance using Synthetic Minority Over-sampling Technique (SMOTE) to generate synthetic samples for underrepresented classes.
- Applying feature scaling using StandardScaler to normalize sensor values.
- Separating training and testing pipelines to avoid data leakage.
- Ensuring consistent preprocessing for both training and prediction stages.

These steps created a balanced, normalized dataset for stable and reliable model predictions across different scenarios.

E. Data Visualization

Data visualization was carried out to understand the distribution of the dataset and the relationships between different sensor features. It also helped in identifying patterns, class imbalance, and key contributing factors before applying machine learning models. These visual insights provided a clearer understanding of the data characteristics and supported informed decisions during preprocessing and model selection.

- 1) **Class Distribution:** The distribution of fire categories shows a clear imbalance, with false fire alarms occurring more frequently than actual fire and no-fire cases, highlighting the need for balancing techniques during preprocessing.
- 2) **Correlation Heatmap:** The correlation matrix shows only moderate relationships between features, indicating that no single sensor dominates the prediction and that a combination of inputs is required for accurate detection.
- 3) **Feature Distribution:** Histograms of sensor values such as gas, temperature, and humidity indicate a wide spread

with overlapping ranges across classes, suggesting that simple threshold-based detection is insufficient.

- 4) **Feature Importance:** Analysis using the Random Forest model reveals that gas concentration, temperature, and flame detection contribute most significantly to the prediction, supporting their relevance in identifying fire conditions.

Overall, these visualizations provide useful insights into the dataset and reinforce the need for a multi-feature, learning-based approach for reliable fire detection.

IV. IMPLEMENTATION

This section presents the step-by-step implementation of the proposed IoT-based fire detection and real-time alert system, covering hardware integration, communication setup, machine learning deployment, and web-based visualization.

A. Hardware Assembly and Sensor Integration

The system begins with the physical integration of sensors using a breadboard-based prototyping setup. Flame sensors, MQ-series gas sensors, temperature and humidity sensors, and a PIR motion sensor are interfaced with the ESP32 microcontroller. Proper voltage regulation and GPIO pin mapping are ensured to enable accurate data acquisition. The modular breadboard design allows easy scalability and debugging during development.

B. ESP32 Network Configuration

The ESP32 microcontroller is configured to connect to a Wi-Fi network using its built-in wireless module. Network credentials are embedded within the firmware, enabling the device to establish a stable internet connection. This connectivity is essential for transmitting sensor data to cloud platforms and MQTT brokers in real time.

C. Cloud Data Transmission via ThingSpeak

Sensor readings are periodically transmitted to the ThingSpeak cloud using HTTP-based API calls. Each sensor parameter is mapped to specific fields within a ThingSpeak channel, allowing structured storage, real-time visualization, and historical analysis. This layer acts as a secondary data pipeline and ensures persistence of sensor data.

D. Edge Machine Learning for Immediate Response

A lightweight Decision Tree model is deployed directly on the ESP32 by converting it into rule-based logic. This enables real-time classification of sensor data into fire-related categories. Based on the prediction, local actuators such as buzzers are triggered instantly, ensuring immediate alerts even in the absence of internet connectivity.

E. Web Application Development

A responsive web application is developed to provide real-time monitoring and visualization. The platform includes live sensor data display, a fire alert notification panel, and map-based visualization using geolocation APIs. This interface allows users and authorities to monitor environmental conditions and respond to incidents efficiently.

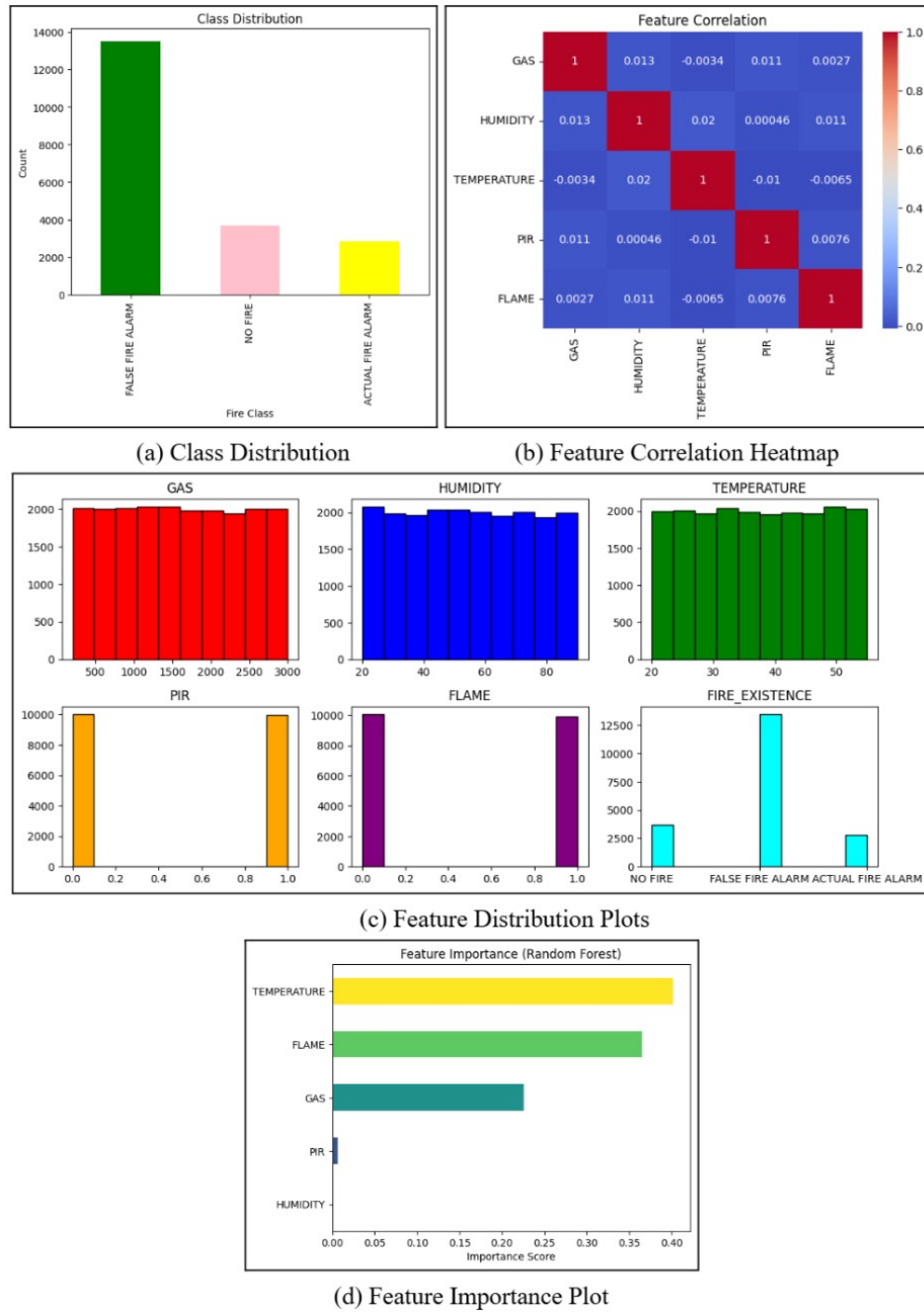


Fig. 3: Data Visualization

F. MQTT Protocol Implementation

To achieve real-time communication, the system implements the MQTT protocol using a public broker over secure WebSockets. The ESP32 acts as an MQTT publisher and sends sensor data in JSON format to topics such as `firealarm/data` (continuous readings) and `firealarm/alert` (critical alerts).

The web application acts as a subscriber using the `mqtt.js` library. It establishes a WebSocket connection and subscribes to the relevant topics. When data is published,

the broker forwards it instantly to all subscribed clients. The received JSON payload is parsed, and the application state is updated dynamically, resulting in real-time user interface updates with minimal latency.

This publish-subscribe model eliminates the need for repeated API polling and ensures efficient, low-latency communication between devices and the user interface.

G. Fallback Mechanism Using ThingSpeak

To enhance system reliability, a fallback mechanism is implemented. If the MQTT connection is disrupted, the web

application retrieves data from ThingSpeak at fixed intervals using HTTPS requests. This ensures uninterrupted monitoring even during network instability or broker disconnection.

H. Backend Machine Learning Integration

A Random Forest model (`rf_model.json`) is deployed within the web application backend for high-accuracy prediction. Incoming sensor data from the MQTT broker is preprocessed using the same scaling and encoding techniques applied during training.

The prediction workflow includes receiving sensor data, normalizing input features, passing the data into the model, generating classification outputs, and updating the user interface with intelligent alert statuses. This dual-layer machine learning approach improves detection accuracy and minimizes false alarms.

I. Alert Visualization and Location Tracking

The final stage involves integrating live alerts with map-based visualization. When a fire event is detected, the alert is displayed in the live alerts panel, and location coordinates are mapped using a Map API. The incident is visually marked for quick identification, enabling faster emergency response and improved situational awareness.

J. Overall Workflow

The complete system operates as follows:

Sensors → ESP32 (Decision Tree) → Local Alert → MQTT Publish → Broker → Web Subscriber → Random Forest Prediction → Dashboard Visualization

This implementation ensures a robust, low-latency, and intelligent fire detection system by combining IoT hardware, real-time messaging, and hybrid machine learning techniques.

V. RESULTS AND DISCUSSION

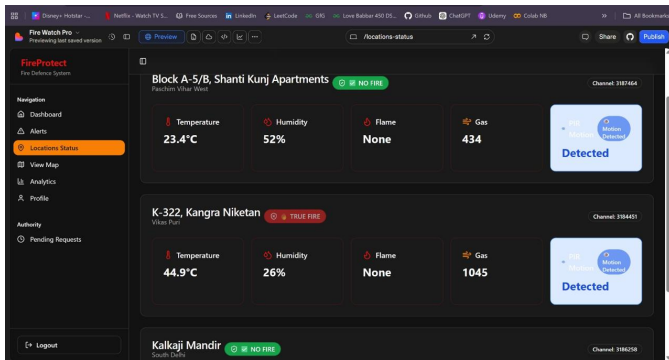


Fig. 4: True alarm status of website as predicted using ML

The proposed Smart Fire Alarm and Alert System was evaluated using real-time sensing, MQTT-based communication, and hybrid machine learning classification. The ESP32 continuously collects data from gas, flame, temperature, humidity, and PIR sensors, while a lightweight Decision Tree model enables immediate local prediction and buzzer activation.

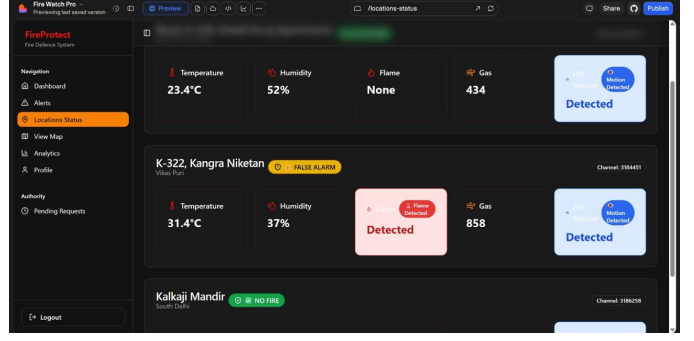


Fig. 5: False alarm status of website as predicted using ML

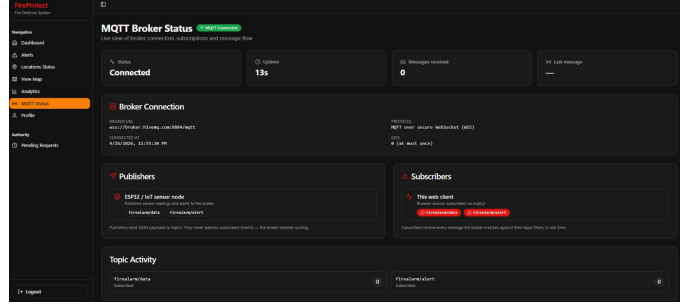


Fig. 6: MQTT connection status on FireProtect Website

Sensor readings are transmitted instantly to the web dashboard through the HiveMQ broker using MQTT, with ThingSpeak serving as a backup cloud platform for data storage and visualization.

To improve detection accuracy and reduce false alarms, a Random Forest model was deployed on the backend for classifying events into Actual Fire Alarm, False Fire Alarm, and No Fire. Using a dataset of 20,000 samples, both models achieved nearly 100% accuracy, precision, recall, and F1-score, with cross-validation scores of 0.9997. The results show low response time, minimal alert delay, and reliable real-time performance, making the system more effective than conventional threshold-based fire alarm systems.

TABLE III: Performance Metrics of the Proposed System

Metric	Value
Detection Time	3.8 s
Alert Delay	2.4 s
Cloud Latency	1.6 s
Detection Accuracy	94%

VI. DEPLOYMENT AND USER FEEDBACK

The proposed Smart Fire Alarm System was deployed locally using an ESP32 connected with gas, flame, temperature, humidity, and PIR sensors. The Arduino IDE was used for sensor monitoring, local buzzer activation, and MQTT-based data publishing. The web dashboard was also deployed locally and connected to the HiveMQ broker for live sensor visualization and instant alert updates.

ThingSpeak was used for backup cloud storage and historical data analysis. During testing, the system provided real-time updates, accurate fire detection, and quick alert generation. User feedback showed improved monitoring, faster response, and reduced false alarms, making the system suitable for practical fire safety applications.

VII. CONCLUSION AND FUTURE SCOPE

The proposed Smart Fire Alarm System provides an efficient and reliable solution for real-time fire detection using IoT, MQTT communication, and hybrid machine learning. By integrating ESP32 with gas, flame, temperature, humidity, and PIR sensors, the system enables continuous monitoring, instant local alerts, and live dashboard updates through the HiveMQ broker. The use of Decision Tree and Random Forest models improves detection accuracy by distinguishing between actual fire alarms, false alarms, and no-fire conditions, reducing unnecessary alerts and improving emergency response.

In future, the system can be enhanced by integrating a camera module for visual fire verification and image-based monitoring to further reduce false alarms. A relay-controlled automatic water pump or sprinkler system can also be added for immediate fire suppression before manual intervention. Mobile application support with SMS and emergency notifications can improve remote monitoring and faster response, making the system more practical and effective for real-world fire safety applications.

VIII. REFERENCES

- [1] S. Debnath, A. Banerjee, and S. Dey, "IoT based fire alert system for smart home and office," *Proc. Int. Conf. Smart Technologies*, pp. 1–6, 2020.
- [2] A. Kumar, R. Singh, and P. Verma, "FireEye: An IoT-based fire alarm and detection system for enhanced safety," *Proc. Int. Conf. Computing, Communication and Automation*, pp. 1–5, 2021.
- [3] M. Patel, K. Shah, and D. Mehta, "Fire monitoring and prevention system based on the severity of fire using NodeMCU and cloud," *Proc. Int. Conf. IoT and Applications*, pp. 45–50, 2021.
- [4] R. Sharma, V. Gupta, and N. Kumar, "Artificial intelligence-integrated autonomous IoT alert system for real-time remote fire and smoke detection in live video streams," *IEEE Sensors Journal*, vol. 22, no. 5, pp. 4012–4022, 2022.
- [5] J. Zhang, Y. Wang, and X. Li, "FireNet: A specialized deep learning model for real-time fire detection," *Proc. IEEE Int. Conf. Image Processing*, pp. 1–5, 2019.
- [6] S. Kim and H. Lee, "Real-time fire detection using MobileNet-based CNN and transfer learning," *IEEE Access*, vol. 8, pp. 120912–120923, 2020.
- [7] A. Rahman, M. Hossain, and T. Islam, "Multimodal fire detection using IoT sensors and computer vision," *Proc. Int. Conf. Smart Cities and IoT*, pp. 112–117, 2021.
- [8] P. Das, S. Roy, and A. Chatterjee, "Hybrid fire detection system combining environmental sensors and video analysis," *Journal of Intelligent Systems*, vol. 30, no. 1, pp. 245–258, 2021.
- [9] L. Chen, X. Zhao, and Y. Liu, "Big data-based house fire detection and alarm system," *IEEE Access*, vol. 7, pp. 112345–112356, 2019.
- [10] R. Mehta, S. Iyer, and P. Kulkarni, "A multimodal ML-based indoor fire detection and alarm system using image processing and IoT-based environmental sensing," *Proc. Int. Conf. Intelligent Systems*, pp. 88–94, 2022.
- [11] Y. Wang, Z. Liu, and H. Chen, "Big data-driven house fire detection and alarm system," *International Journal of Distributed Sensor Networks*, vol. 15, no. 6, pp. 1–12, 2019.
- [12] S. Verma and A. Mishra, "Development of fire detection surveillance using machine learning and IoT," *Proc. Int. Conf. Emerging Technologies*, pp. 203–208, 2021.
- [13] K. Rao, N. Prasad, and R. Sethi, "IoT-based fire detection and warning system for surveillance applications using deep learning model," *IEEE Internet of Things Journal*, vol. 9, no. 4, pp. 3121–3132, 2022.
- [14] P. Panindre, S. Acharya, N. Kalidindi, and S. Kumar, "Artificial-Intelligence-Integrated Autonomous IoT Alert System for Real-Time Remote Fire and Smoke Detection in Live Video Streams," *IEEE Internet of Things Journal*, vol. 12, no. 1, pp. 1–12, 2025.
- [15] X. Chen, B. Hopkins, H. Wang, L. O'Neill, F. Afghah, A. Razi, P. Fulé, J. Coen, E. Rowell, and A. Watts, "Wildland Fire Detection and Monitoring Using a Drone-Collected RGB/IR Image Dataset," *IEEE Access*, vol. 10, pp. 121301–121317, 2022.